

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS
CAMPUS TAMPICO – FACULTAD DE INGENIERÍA ARTURO NARRO
SILLER

Proyecto de Semestre

Analizador Léxico para Nomenclatura Química

Lenguaje Desarrollado: LexQuimica

Materia: Programación de Sistemas de Base 1
Profesor: Dr. Dante Adolfo Muñoz Quintero
Grupo: 8°M
Semestre: 2026-1
Repositorio: <https://github.com/SergioAlejandre/analizador-lexico-lexquimica.git>
Fecha de Entrega: 16 de mayo de 2026

Integrantes:

Perez Castan Alejandro Emmanuel matrícula:2223330182
Sanchez Garcia Gael Antonio matrícula:2223330195
Alejandre Mar Sergio Adrian matrícula:2223330133

Resumen

El presente documento constituye la Demostración Funcional del proyecto de semestre para la materia de Programación de Sistemas de Base 1. Incluye la especificación del lenguaje diseñado, el código fuente comentado del analizador léxico y capturas de ejecución con casos válidos y con errores. El sistema se centra en el lenguaje **LexQuímica**, desarrollado en Python 3 utilizando la herramienta ANTLR v4 para transformar archivos de texto con nomenclatura química sistemática en flujos de tokens estructurados, reportando con precisión matemática las coordenadas por línea y columna de cualquier anomalía detectada.

Tabla de contenido

1	Introducción	3
2	Objetivos	4
2.1	Objetivo general	4
2.2	Objetivos específicos	4
3	Especificación del Lenguaje	5
3.1	Nombre y propósito	5
3.2	Ejemplos ilustrativos de código	5
3.3	Catálogo de tokens	5
4	Implementación	6
4.1	Definición de tokens y palabras reservadas	6
4.2	Reglas léxicas	6
4.3	Lógica del analizador léxico	7
4.4	Manejo y reporte de errores	7
5	Demostración Funcional	9
5.1	Ejecución con programa válido	9
5.2	Ejecución con errores léxicos	10
5.3	Caso con identificadores inválidos y símbolos desconocidos	11
6	Conclusiones	13
7	Trabajo futuro	14
8	Referencias	15

1 Introducción

El análisis léxico es la primera fase de un compilador o intérprete: recibe una secuencia de caracteres y produce una secuencia de tokens que representan las unidades mínimas con significado del lenguaje. Comprender esta etapa de forma práctica es fundamental en la formación de un ingeniero en sistemas computacionales que desee construir herramientas de procesamiento de lenguajes.

El presente proyecto diseña e implementa un analizador léxico para **LexQuímica**, un lenguaje personalizado de propósito educativo orientado a la escritura y parseo de compuestos químicos estructurados bajo la nomenclatura sistemática (IUPAC). El nombre refleja su propósito principal: un analizador léxico funcional especializado en nomenclaturas de la disciplina química.

El lexer fue implementado utilizando la potencia generadora de la herramienta ANTLR v4 combinado con un script conductor en Python 3. Se diseñó específicamente para transformar archivos de texto fuente en una lista numerada de tokens estructurados, así como para informar con precisión la línea y columna de cualquier error léxico o carácter no permitido que se detecte durante el proceso de escaneo continuo del buffer.

2 Objetivos

2.1 Objetivo general

Diseñar un lenguaje personalizado denominado **LexQuimica** e implementar un analizador léxico funcional en Python mediante ANTLR v4 que transforme programas fuente escritos en dicho lenguaje en listas organizadas de tokens, con un manejo preciso y robusto de errores léxicos coordinados.

2.2 Objetivos específicos

1. Definir la especificación informal del lenguaje **LexQuimica** mediante ejemplos ilustrativos de código fuente que cubran sus construcciones e instrucciones principales.
2. Establecer un catálogo completo de tokens divididos por categorías: palabras reservadas, identificadores válidos, operadores de transferencia, delimitadores del sistema y constantes numéricas.
3. Implementar el analizador léxico usando las reglas gramaticales de ANTLR v4, asegurando el reconocimiento correcto de la precedencia y la longitud máxima de coincidencia de los tokens definidos.
4. Implementar la detección y reporte detallado de errores léxicos, indicando el número exacto de línea, columna y descripción formateada del problema para cada anomalía encontrada en el archivo fuente.
5. Probar el lexer con un archivo mixto que combine flujos de compuestos químicos válidos con casos que provoquen fallos léxicos intencionales para evaluar la resiliencia del sistema de control.

3 Especificación del Lenguaje

3.1 Nombre y propósito

LexQuimica es un lenguaje imperativo de dominio específico diseñado con un propósito estrictamente educativo para expresar asignaciones de variables de datos y declaraciones secuenciales de compuestos químicos inorgánicos. La extensión de los archivos de texto utilizados como código fuente de entrada para este sistema traductor es `.txt`.

3.2 Ejemplos ilustrativos de código

A continuación se presentan los ejemplos que ilustran la sintaxis correcta del lenguaje para declarar compuestos químicos sistemáticos válidos y asignarlos a identificadores del sistema:

```
1 sustancia_A = trioxido de dihierro;
2 resultado = monoxido de carbono;
```

Código Fuente 1: Ejemplos de sentencias válidas en LexQuimica

3.3 Catálogo de tokens

La Tabla 1 expone el catálogo oficial de componentes atómicos reconocidos por el analizador, mapeando cada elemento a su respectiva categoría semántica y patrón operativo.

Tabla 1: Catálogo Estructurado de Tokens de LexQuimica

Categoría	Ejemplos	Descripción
P_Reservada	oxido, de, mono, di, tri, hierro, cloro	Palabras reservadas del lenguaje
ID	sustancia_A, resultado, mi_variable	Identificadores válidos (letras/guion bajo)
OPERADOR	=	Símbolo de asignación de datos
DELIMITADOR	;, ,, (,), {, }	Separadores y agrupadores del sistema
NUMERO	42, 1024, 3.14, 0.5	Constantes numéricas enteras y flotantes
ID_INVALIDO	123error, 5sustancia	Identificadores erróneos que inician con dígito

4 Implementación

A continuación se presenta el código fuente de los módulos centrales del analizador léxico desarrollado para este proyecto de sistemas base.

4.1 Definición de tokens y palabras reservadas

Las palabras reservadas de LexQuimica corresponden a términos predefinidos de la disciplina química que el sistema bajo ningún concepto debe clasificar como identificadores de variables genéricas. En la infraestructura de ANTLR v4, esto se logra ordenando jerárquicamente las reglas específicas por encima de los patrones comodín.

4.2 Reglas léxicas

Las reglas que gobiernan el comportamiento de reconocimiento léxico del lenguaje se especifican formalmente dentro del archivo de gramática con extensión `.g4`:

```
1 grammar Quimica;
2
3 // --- REGLAS DEL PARSER ---
4 programa : instruccion+ EOF ;
5 instruccion : asignacion | compuesto PUNTO_COMA ;
6 asignacion : ID ASIGNACION compuesto PUNTO_COMA ;
7 compuesto : prefijo? FUNCION CONECTOR prefijo? ELEMENTO ;
8 prefijo    : MONO | DI | TRI | TETRA | PENTA | HEXA | HEPTA ;
9
10 // --- REGLAS DEL LEXER ---
11 FUNCION    : 'oxido' | ' xido ' | 'xido' | 'hidruro' | 'hidroxido' | '
    hidr xido' ;
12 CONECTOR   : 'de' ;
13
14 MONO       : 'mono' ;
15 DI         : 'di' ;
16 TRI        : 'tri' ;
17 TETRA      : 'tetra' ;
18 PENTA      : 'penta' ;
19 HEXA       : 'hexa' ;
20 HEPTA      : 'hepta' ;
21
22 ELEMENTO    : 'hierro' | 'sodio' | 'calcio' | 'magnesio' | 'aluminio' | '
    carbono' | 'cloro' ;
23
24 ASIGNACION  : '=' ;
25 PUNTO_COMA : ';' ;
26 COMA       : ',' ;
27 PAREN_IZQ  : '(' ;
28 PAREN_DER  : ')' ;
29 LLAVE_IZQ  : '{' ;
30 LLAVE_DER  : '}' ;
31
32 ID : [a-zA-Z_][a-zA-Z0-9_]* ;
```

```

33 ID_INVALIDO : [0-9]+[a-zA-Z_][a-zA-Z0-9_]* ;
34 NUMERO_FLOAT : [0-9]+ '.' [0-9]+ ;
35 NUMERO_INT : [0-9]+ ;
36
37 WS : [ \t\r\n]+ -> skip ;
38 ERROR : . ;

```

Código Fuente 2: Especificación de Reglas Léxicas en Quimica.g4

4.3 Lógica del analizador léxico

La lógica de transiciones y procesamiento secuencial es generada de manera automática por ANTLR en la clase interna `QuimicaLexer.py`, aplicando internamente la regla de coincidencia más larga para extraer los lexemas del buffer de caracteres sin fragmentaciones erróneas.

4.4 Manejo y reporte de errores

El script de control principal `main.py` actúa como el módulo de manejo de errores, interceptando tokens clasificados como `ERROR` o `ID_INVALIDO` y calculando en tiempo real su fila y columna:

```

1 import sys
2 from antlr4 import *
3 from QuimicaLexer import QuimicaLexer
4
5 def main():
6     nombre_archivo = sys.argv[1] if len(sys.argv) > 1 else "quimica.txt"
7     try:
8         input_stream = FileStream(nombre_archivo, encoding='utf-8')
9         lexer = QuimicaLexer(input_stream)
10        token = lexer.nextToken()
11
12        tokens_validos = []
13        errores_detectados = []
14
15        while token.type != Token.EOF:
16            nombre_token = lexer.symbolicNames[token.type]
17
18            if nombre_token == "ERROR":
19                errores_detectados.append(
20                    f"Linea {token.line}, Columna {token.column}: Símbolo
21                    desconocido '{token.text}' detectado."
22                )
23            elif nombre_token == "ID_INVALIDO":
24                errores_detectados.append(
25                    f"Linea {token.line}, Columna {token.column}:
26                    Identificador inválido '{token.text}'."
27                )
28            elif nombre_token != "WS":
29                categoria = nombre_token
30                if nombre_token in ["FUNCION", "CONECTOR", "MONO", "DI", "
31                TRI", "TETRA", "PENTA", "HEXA", "HEPTA", "ELEMENTO"]:
```

```
29         categoria = "P_Reservada"
30     elif nombre_token == "ID":
31         categoria = "ID"
32     elif nombre_token == "ASIGNACION":
33         categoria = "OPERADOR"
34     elif nombre_token in ["PUNTO_COMA", "COMA", "PAREN_IZQ", "
PAREN_DER", "LLAVE_IZQ", "LLAVE_DER"]:
35         categoria = "DELIMITADOR"
36     elif nombre_token in ["NUMERO_INT", "NUMERO_FLOAT"]:
37         categoria = "NUMERO"
38
39     tokens_validos.append((categoria, token.text))
40     token = lexer.next_token()
41
42     print("Token List:")
43     for i, (cat, val) in enumerate(tokens_validos, start=1):
44         print(f" {i}: ({cat}, \"{val}\")")
45
46     print("\nError List:")
47     if errores_detectados:
48         for err in errores_detectados:
49             print(f" {err}")
50     else:
51         print(" No lexical errors found.")
52 except Exception as e:
53     print(f"Error general al procesar el archivo: {e}")
54
55 if __name__ == '__main__':
56     main()
```

Código Fuente 3: Módulo Conductor de Control main.py

5 Demostración Funcional

Las siguientes secciones ilustran las pruebas y evidencias gráficas recopiladas desde la terminal del sistema operativo durante las fases de validación funcional.

5.1 Ejecución con programa válido

Se procesa el archivo fuente que contiene únicamente sentencias correctas y compuestos bien estructurados. El analizador emite el listado de tuplas sin registrar incidencias léxicas.

```
Lista de Tokens:
1: (ID, "sustancia_A")
2: (OPERADOR, "=")
3: (ID, "trioxido")
4: (P_Reservada, "de")
5: (ID, "dihierro")
6: (DELIMITADOR, ";")
7: (ID, "resultado")
8: (OPERADOR, "=")
9: (ID, "monoxido")
10: (P_Reservada, "de")
11: (P_Reservada, "carbono")
12: (DELIMITADOR, ";")
13: (ID, "compuesto_tres")
14: (OPERADOR, "=")
15: (ID, "heptaoxido")
16: (P_Reservada, "de")
17: (ID, "dicloro")
18: (DELIMITADOR, ";")

Lista de Errores:
No se encontraron errores lexicos.
```

Figura 1: Tokenización completa del archivo de compuestos químicos sin errores léxicos.

5.2 Ejecución con errores léxicos

Se somete a evaluación un archivo que incorpora caracteres explícitamente prohibidos por el alfabeto del lenguaje (tales como \$ o @), demostrando que el lexer los aísla de forma inmediata.

```
Lista de Tokens:
1: (OPERADOR, "=")
2: (ID, "heptaxido")
3: (P_Reservada, "de")
4: (ID, "dicloro")
5: (DELIMITADOR, ";")
6: (ID, "sustancia_B")
7: (OPERADOR, "=")
8: (ID, "trioxido")
9: (P_Reservada, "de")
10: (P_Reservada, "hierro")
11: (DELIMITADOR, ";")
12: (ID, "compuesto_falla")
13: (OPERADOR, "=")
14: (ID, "monoxido")
15: (P_Reservada, "de")
16: (P_Reservada, "carbono")
17: (ID, "quimica")
18: (OPERADOR, "=")
19: (P_Reservada, "oxido")
20: (P_Reservada, "de")
21: (P_Reservada, "sodio")
22: (DELIMITADOR, ";")

Lista de Errores:
Linea 1, Columna 0: Identificador inválido '123error'.
Linea 3, Columna 38: Símbolo desconocido '$' detectado.
Linea 4, Columna 0: Símbolo desconocido '@' detectado.
```

Figura 2: El lexer detecta los símbolos inválidos, reporta su ubicación exacta y continúa procesando.

5.3 Caso con identificadores inválidos y símbolos desconocidos

Esta prueba evalúa la resiliencia combinada de la máquina de estados al ingresar identificadores que quebrantan la regla de formación al iniciar con un dígito numérico continuo. El sistema separa la anomalía y reporta con precisión milimétrica la columna de origen del fallo.

```
Lista de Tokens:
1: (OPERADOR, "=")
2: (P_Reservada, "oxido")
3: (P_Reservada, "de")
4: (P_Reservada, "sodio")
5: (DELIMITADOR, ";")
6: (ID, "sustancia_C")
7: (OPERADOR, "=")
8: (ID, "heptaoxido")
9: (P_Reservada, "de")
10: (ID, "dicloro")
11: (DELIMITADOR, ";")
12: (ID, "compuesto")
13: (OPERADOR, "=")
14: (ID, "trioxido")
15: (P_Reservada, "de")
16: (ID, "dihierro")

Lista de Errores:
Linea 1, Columna 0: Identificador inválido '99hierro'.
Linea 3, Columna 33: Símbolo desconocido '#' detectado.
```

Figura 3: Detección de identificadores que comienzan con dígito y símbolos no reconocidos.

6 Conclusiones

- **Perez Castan Alejandro Emmanuel:** Diseñar la tabla de tokens y las expresiones regulares asociadas me permitió asimilar en la práctica por qué el orden de jerarquía de las reglas léxicas es crítico en ANTLR. Comprender cómo se comportan las palabras clave frente a identificadores comunes enriqueció mi visión del software base.
- **Sanchez Garcia Gael Antonio:** El mayor desafío técnico del proyecto consistió en estructurar el manejo de errores léxicos coordinados de tal forma que una falla horizontal no detuviera abruptamente el análisis del resto del archivo de texto. Implementar la rutina de recuperación proporciona un módulo de diagnóstico de gran utilidad para el usuario final.
- **Alejandro Mar Sergio Adrian:** Separar limpiamente las reglas gramaticales en el archivo `.g4` de la lógica de clasificación y traducción cosmética en el archivo `main.py` facilitó de gran manera el ciclo de validación de pruebas de laboratorio. Redefinir el enfoque del proyecto eliminando APIs de simulación externas nos permitió concentrarnos en la esencia algorítmica de la materia.

7 Trabajo futuro

- Extender las capacidades del analizador léxico para reconocer las nomenclaturas químicas de tipo **Stock** (reconocimiento e interpretación de valencias mediante números romanos) y **Tradicional** (detección de sufijos estequiométricos de electrones).
- Desarrollar e integrar el analizador sintáctico (*parser*) completo sobre las clases generadas para validar formalmente la estructura gramatical de las sentencias del lenguaje.
- Diseñar un módulo de análisis semántico que cruce las valencias de los compuestos para validar de forma matemática la neutralidad de cargas de las moléculas introducidas.

8 Referencias

1. Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.a ed.). Pearson Addison-Wesley.
2. Cooper, K. D. y Torczon, L. (2011). *Engineering a Compiler* (2.a ed.). Morgan Kaufmann.
3. Levine, J. (2009). *Flex & Bison: Text Processing Tools*. O'Reilly Media.
4. Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
5. Python Software Foundation. (2024). *re – Regular expression operations*. Recuperado de <https://docs.python.org/3/library/re.html>